

A Study on the Dynamics and Effectiveness of the Deflate Geometric Semantic Mutation

Davide Farinati, Gloria Pietropolli, Leonardo Vanneschi

Abstract—Geometric Semantic Genetic Programming (GSGP) is a variant of Genetic Programming (GP) that induces an error surface without local minima for supervised learning tasks. However, GSGP is limited by the fact that its operators produce increasingly large individuals, leading to overly complex models. The Semantic Learning algorithm based on Inflate and deflate Mutation (SLIM) addresses this issue by introducing a deflate geometric semantic mutation capable of producing offspring smaller than their parents. Preliminary studies show that SLIM can maintain accuracy comparable to traditional GSGP while generating much smaller models. However, a thorough analysis of this mutation remains lacking. This work fills that gap by conducting a detailed study of the deflate mutation, focusing on its behavior and practical value. Our results show that, when applied at the right stage of evolution, deflate mutation mitigates overfitting and yields compact, accurate models. This is also the first study to explore the timing and interaction of inflate and deflate mutations in SLIM, demonstrating how deflation enhances generalization and reduces overfitting. We support our conclusions with a comprehensive experimental approach, including comparisons between exclusive use of inflate mutation and alternating it with deflation. We also evaluate numerical indicators such as improvement rate and training effectiveness. The consistency across these methods reinforces our findings and highlights the deflate mutation as a robust regularization strategy. Finally, when compared to established non-evolutionary machine learning methods, SLIM shows competitive performance. Overall, this study confirms SLIM as a promising direction for GP and lays the foundation for future research.

Index Terms—Genetic Programming, Geometric Semantic Genetic Programming, Mutation, Deflate Mutation

I. INTRODUCTION

Geometric Semantic Genetic Programming (GSGP) [1] is a variant of Genetic Programming (GP) [2, 3, 4] that employs Geometric Semantic Operators (GSOs) instead of the traditional GP crossover and mutation. GSOs have the advantage of inducing a unimodal error surface, but they have the drawback of generating large individuals. Over the past decade, GSGP has gained popularity, spurring extensive research [5, 6, 7, 8, 9, 10] and diverse applications [11, 12]. Recently, a new GSGP variant, the Semantic Learning algorithm based on Inflate and deflate Mutation (SLIM) [13, 14], has introduced the Deflate Geometric Semantic Mutation (DGSM). Contrary to traditional geometric semantic mutation (which we call Inflate Geometric Semantic Mutation (IGSM)), DGSM generates

offspring that are smaller than their parents, while retaining the beneficial properties of GSOs. Preliminary work [13, 14] indicates that SLIM can maintain the same good performance as traditional GSGP, while at the same time generating models that are much smaller and suitable for human interpretation. However, beyond these initial studies, there is a significant lack of in-depth exploration of the dynamics and utility of DGSM.

Building on the foundational research that introduced SLIM [13, 14], this work conducts a comprehensive analysis of DGSM. This study is structured to investigate the dynamics of DGSM, in an attempt to validate three hypotheses: (i) DGSM can generate individuals of smaller size than traditional GSGP, without compromising performance; (ii) there are some stages of the evolution where DGSM proves to be most effective; and (iii) DGSM can effectively contribute to preventing overfitting, ensuring robust model performance. To solidify the validity of our conclusions, we seek confirmation of these hypotheses through a variety of studies and from different perspectives.

As a first step, we establish an experimental framework in which IGSM and DGSM are used intermittently throughout various generations. The process starts with the exclusive use of IGSM, allowing the individuals in the population to grow in size. At a predetermined generation, the operator is switched to exclusively DGSM, which then allows for a reduction in the size of the individuals. This approach is novel compared to [13, 14], where, instead, IGSM and DGSM had always been used simultaneously. Through this approach, we can effectively isolate the effects of IGSM and DGSM to gain a clearer understanding of their respective impacts on the algorithm. The experimental setup is designed to compare a version that exclusively uses IGSM throughout the entire run (which corresponds to traditional GSGP) with multiple versions where DGSM is introduced at various stages. Initially, we evaluate the performance of these variants by examining the plots showing the evolution of the RMSE and the size of the individuals. However, to gain a more quantitative understanding that extends beyond simply observing the plots, we analyze the same experiments using the Improvement Rate (IR). We specifically introduce IR as a measure to quantify the comparative advantage of one algorithm over another. The IR is applied here to assess the benefits of different versions that integrate DGSM at various stages against the version that uses only IGSM. This approach provides a clear assessment of the tangible benefits of incorporating DGSM and helps identify the most effective stages in the evolutionary process for its application.

As a second set of experiments, we evaluate the effective-

D. Farinati and L. Vanneschi are with the NOVA Information Management School (NOVA IMS), Universidade Nova de Lisboa, Campus de Campolide, 1070-312 Lisboa, Portugal.

G. Pietropolli is with the Department of Mathematics, Informatics and Geosciences (DMG), University of Trieste, Via Alfonso Valerio 12/1, 34128 Trieste, Italy (e-mail: gloria.pietropolli@units.it).

Corresponding author: D. Farinati (dfarinati@novaims.unl.pt).

ness of DGSM, compared to IGSM, utilizing a measure known as Training Improvement Effectiveness (TIE) [15]. TIE examines the semantic neighborhood of a given individual, induced by a specific mutation. Essentially, the concept underlying the TIE for an individual x is that the count of semantic neighbors of x fitter than x can provide insights into the mutation’s potential effectiveness in subsequent generations. Originally introduced as a criterion for early stopping [15], TIE has more recently been employed also to manage populations of dynamic size [16]. In this study, we compare the TIE scores of IGSM and DGSM along the evolution, to corroborate the previous results concerning their relative effectiveness at different evolutionary stages.

In synthesis, compared to previous work on SLIM, this study introduces the following elements of novelty: (1) a new perspective on the use of inflate and deflate mutations in SLIM, (2) a comprehensive experimental framework, incorporating new quantitative indicators such as IR and TIE, and (3) a comparison of SLIM with commonly used machine learning algorithms, aimed at evaluating the true potential of SLIM, particularly in terms of generalization capability.

Point (1) focuses on identifying the most effective phase for applying deflate mutation, investigating for the first time the timing and interaction of variation operators, offering new insights into how their coordination can improve the evolutionary search process. Operators are no longer viewed as static and additive components, but as interacting mechanisms whose effect depends on when they are applied. Point (2) strengthens the conclusions of Point (1), also proposing a novel use of TIE, that can be extended to other different studies. Point (3) includes in the experimental comparison with SLIM not only standard GP and traditional GSGP, but also other commonly used, non-evolutionary machine learning algorithms, such as Linear Regression, Decision Trees, Support Vector Machines, and Multi-layer Perceptron.

II. BACKGROUND AND OPERATOR DEFINITIONS

This section starts by introducing the standard version of GSGP [1] and SLIM [13, 14]. Then, it introduces the concepts of Improvement Rate (IR), Semantic Neighborhood (SN) and finally the Training Improvement Effectiveness (TIE) [15], that will be used later to study the impact of DGSM.

A. Geometric Semantic Genetic Programming

GP is an evolutionary method that evolves programs to map input examples $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ to target outputs [2]. When a program P processes $\mathbf{X}$, it produces a semantic vector $s_P = [P(\mathbf{x}_1), P(\mathbf{x}_2), \dots, P(\mathbf{x}_n)]$, representing its behavior on the input set [1]. This vector is referred to as the semantics of the program. Traditional GP operators, such as one-point crossover and subtree mutation [2], modify the genotype structure of programs, but their impact on semantics is typically unpredictable.

GSGP [1] is a variant of GP characterized by the use of GSOs—operators that produce predictable and structured changes on the semantics of the individuals. These operators generate unimodal error surfaces for supervised learning tasks [17, 18], improving search efficiency. Studies have shown

that mutation-only GSGP often performs as well as or better than configurations using both crossover and mutation [19, 20], leading us to focus on Geometric Semantic Mutation (GSM).

For symbolic regression, given a parent function $T : \mathbb{R}^n \rightarrow \mathbb{R}$ and a mutation step ms , GSM produces:

$$GSM(T) = T + ms \cdot (T_{R1} - T_{R2}),$$

where T_{R1} and T_{R2} are randomly generated functions.

GSM perturbs the semantics of T by adding a zero-centered random component. Since functions typically return different values for different inputs, this ensures a distinct perturbation for each input point. Using the difference $T_{R1} - T_{R2}$ centers the perturbation distribution at zero, making positive and negative changes equally likely.

Although not in the original definition [1], subsequent work [18, 20, 21] has shown that constraining the outputs of T_{R1} and T_{R2} into an interval like $[0, 1]$ – e.g., using a sigmoid function – can enhance generalization. In this way, GSM introduces semantic perturbations bounded within $[-ms, ms]$, effectively acting as a ball mutation of radius ms in the semantic space. While it provides desirable optimization properties, it also increases the size of the resulting program, as the offspring includes the genotypes of T , T_{R1} , T_{R2} , plus other structural elements.

B. Different Definitions of a Ball Mutation

From now on, the GSM formulation described in the previous section, where the codomains of the random expressions T_{R1} and T_{R2} are restricted to the interval $[0, 1]$ using a sigmoid function, will be referred to as 2SIG (which stands for “sigmoid with two random expressions”). In other words: $2SIG = ms \cdot (S(T_{R1}) - S(T_{R2}))$, and S is the sigmoid function.

However, 2SIG is only one of the numerous ways one could imagine to define a ball mutation with radius ms . Several other functions were introduced for this aim [13, 14, 22]. Here, in addition to 2SIG, we will focus on the following approaches:

- 1SIG (which stands for “sigmoid with one random expression”), where: $1SIG = ms \cdot (2 \cdot S(T_R) - 1)$.
- ABS, (which stands for “absolute value”), where: $ABS = ms \cdot \left(1 - \frac{2}{1+T_R}\right)$.

Although 1SIG and ABS have different geometries (studied in depth in [14]) compared to 2SIG, they both have the advantage of using one random expression instead of two, thereby allowing us to save computational resources.

In the literature [13, 14, 18] the definition of the GSM operator has been rewritten as: $GSM(T) = T + \Delta(T)$, where $\Delta(T)$ is the function used to induce a ball mutation with radius ms on the semantic space. Transversal to the different approaches for the definition of $\Delta(T)$, one could also imagine two different ways of inducing a perturbation to the value of a given expression T : adding to T a value that is either slightly smaller or slightly bigger than zero, or multiplying T by a value that is either slightly smaller or slightly bigger than 1. In other words, for any of the $\Delta(T)$ variants discussed above, besides a mutation implemented as $GSM(T) = T + \Delta(T)$, we could also imagine a mutation defined as: $GSM(T) = T \cdot (1 + \Delta(T))$.

Combining these three ways of defining $\Delta(T)$ with these two ways of inducing a perturbation to T , we obtain six different variants, that were studied in [13, 14]. Here, we focus on the variants that in previous works [14] demonstrated the most promise, i.e.: (1) SLIM+2SIG, where mutation is defined as $GSM(T) = T + 2SIG$; (2) SLIM*1SIG, where mutation is defined as $GSM(T) = T \cdot (1 + 1SIG)$; and (3) SLIM*ABS, where mutation is defined as $GSM(T) = T \cdot (1 + ABS)$.

C. Semantic Learning algorithm based on Inflate and deflate Mutation

SLIM [13] is an extension of GSGP that, besides the geometric semantic mutation previously discussed, also uses a novel mutation operator. Similar to the previously discussed one, this new mutation has the effect of inducing a ball mutation of radius ms on the semantic space. But contrarily to the previous one, this new mutation generates offspring that are smaller than their parents. For this reason, this new mutation is called Deflate Geometric Semantic Mutation (DGSM) and, in contrast, the previously discussed operator is often referred to as Inflate Geometric Semantic Mutation (IGSM).

Two intuitions inspired the definition of DGSM: first, the definition of GSM given previously can be rewritten as $GSM(T) = T + ms \cdot (T_{R1} - T_{R2}) = T - ms \cdot (T_{R2} - T_{R1})$; second, the random trees T_{R1} and T_{R2} are interchangeable, as they are independently sampled from the same distribution. So, the definition of GSM is equivalent to $GSM(T) = T - ms \cdot (T_{R1} - T_{R2})$.

To clarify why this observation becomes crucial to define an operator able to reduce the size of the individuals, let us now consider as an example an individual T on which GSM is applied, say, three consecutive times. The result of this operation is: $T_M = GSM^3(T) = T + ms \cdot (T_{R1} - T_{R2}) + ms \cdot (T_{R3} - T_{R4}) + ms \cdot (T_{R5} - T_{R6})$. On the other hand, if we apply GSM once more, but this time using subtraction instead of addition, the generated individual is: $T_{M'} = T + ms \cdot (T_{R1} - T_{R2}) + ms \cdot (T_{R3} - T_{R4}) + ms \cdot (T_{R5} - T_{R6}) - ms \cdot (T_{R7} - T_{R8})$.

It is now important to recall that a widely adopted approach in the literature is to reuse random trees instead of generating new ones [5]. For example, we could use T_{R3} and T_{R4} instead of T_{R7} and T_{R8} , leading to: $T_{M'} = T + ms \cdot (T_{R1} - T_{R2}) + ms \cdot (T_{R3} - T_{R4}) + ms \cdot (T_{R5} - T_{R6}) - ms \cdot (T_{R3} - T_{R4})$. So, after simplification, we have $T_{M'} = T + ms \cdot (T_{R1} - T_{R2}) + ms \cdot (T_{R5} - T_{R6})$, which has a smaller genotype than T_M . Using this idea, DGSM simply works by removing one of the terms that IGSM had previously added to an individual, effectively performing a geometric semantic mutation.

Also, it is worth pointing out that $T_{M'}$ is not the product of a backtracking operation, as the individual was not present in any of the previous steps.

In our previous example, we have used the SLIM+2SIG variant, but the same reasoning also holds for all the other discussed variants. In particular, for SLIM*1SIG and SLIM*ABS we have to imagine that the deflate mutation divides by a previously multiplied term, thus leading to a simplification of the genotype.

In the implementation of SLIM, it is helpful to visualize

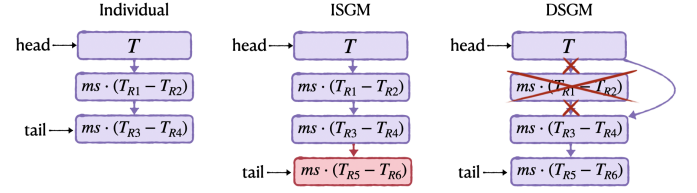


Fig. 1: An example of a SLIM+2SIG individual, with a visual representation of the effect of the IGSM and DGSM on its genotype. When the IGSM is applied, a block is appended, while the DGSM removes a block.

the genotype of each individual as a *linked list*, in which the first element is a random expression created in the population initialization phase, and every following element is the genetic material added by the IGSM, one item for each IGSM application, as shown in Figure 1. DGSM acts on the genotype by removing a random item (different from the first one) from the linked list. To calculate the semantics of the whole individual, it is enough to sum (in the case of SLIM+2SIG) or multiply (in the cases of SLIM*1SIG and SLIM*ABS) the semantics of the different items in the list. The DGSM definition for these three different variants of SLIM is given in the Supplementary Material.

D. Improvement Rate

When comparing different algorithms, the plots depicting the evolution of the RMSE or the size of the individuals provide a visual representation of their respective performance. This visual analysis represents our initial approach for comparing the different SLIM variants; however, it does not quantify one algorithm's competitive advantages over the others. Thus, to provide a more quantitative analysis of the differences between the compared SLIM versions, we also include in our experiments a study of the Improvement Rate of fitness (IR_f) and the Improvement Rate of size (IR_s). These rates are defined as: $IR_f = \frac{f_d - f_i}{f_i}$ and $IR_s = \frac{s_d - s_i}{s_i}$ where f_d represents the fitness at the end of the evolution of one of the variants that switch to DGSM and s_d denotes the size of its elite at termination, while f_i refers to the fitness at the end of the evolution of the variant that only uses IGSM, with s_i indicating the size of its elite at termination. f_d , f_i , s_d and s_i are all medians calculated over 30 independent runs, as we will clarify in Section II-E. The improvement rate provides a numerical quantification of the benefits or drawbacks associated with switching to DGSM in terms of fitness (represented by IR_f) and model size (represented by IR_s). A positive value of the improvement rate indicates that the version that switches to DGSM is preferable compared to the version that maintains IGSM. An improvement rate equal to zero indicates a tie between these two versions. A negative improvement rate indicates that the version that maintains IGSM is preferable compared to the version that switches to DGSM.

E. Training Improvement Effectiveness

The Semantic Neighborhood (SN), first introduced in [15], refers to the set of individuals that can be reached from a given reference individual when a specific GSM is applied. Based on the concept of SN, the TIE, also introduced in [15], refers to the percentage of instances where the GSM produces a better neighbor compared to the reference individual. The

TIE is a powerful tool to get information regarding the effectiveness of the search process dynamically during the evolution. Following [15, 16], in this work we always consider the SN built using as reference the best individual in the population at a given generation, from now on referred to as the elite. Furthermore, given that the cardinality of the SN is generally huge, as in [15, 16], we approximate the calculation of the TIE by sampling the SN considering several randomly generated elite neighbors equal to the population size. In this way, the TIE is calculated by counting the number of neighbors with a better fitness than the elite in this sample, normalized by the sample size.

A comparison of the evolution of the TIE obtained applying DGSM and the one obtained applying IGSM can be very informative about the differences between these two operators at different evolution stages. We point out that, when building the SN using the IGSM, it is always possible to create m different neighbors, with m being the SN size. In contrast, when we use the DGSM, the number of possible different neighbors is limited by the elite's structure, specifically by the number of items that compose its linked-list: given a SLIM individual with m items in its list, the DGSM can produce a SN of at most $m - 1$ different neighbors, each obtained by removing one of the items, except for the first one.

III. EXPERIMENTAL SETUP

As test problems, we selected seven symbolic regression tasks commonly used as benchmarks in GP (e.g., [15, 19, 23, 24, 25, 26, 27, 28]). These datasets have been widely adopted in the literature due to their varying characteristics and practical relevance. They differ significantly in both sample size and input dimensionality, leading to differences in learning difficulty and overfitting risk. This diversity enables us to evaluate the behavior of DGSM in a wide range of regression problems. The selected datasets include: yacht (containing 307 observations and 6 features), airfoil (1502 observations and 5 features), concrete slump (102 observations and 9 features), concrete strength (1029 observations and 8 features), ppb (131 observations and 626 features), bioavailability (359 observations and 241 features) and ld50 (234 observations with 626 features). The first group of datasets involves physical modeling problems: Yacht hydrodynamics, airfoil self-noise, concrete slump, and concrete strength. These are relatively low-dimensional and typically exhibit low overfitting risk, making them commonly used in GP benchmarking. The second group – ppb, bioavailability, and ld50 – concerns pharmacokinetic modeling tasks and involves high-dimensional data with limited sample sizes, making them substantially more prone to overfitting, as shown in previous work.

The hyperparameters were determined through a preliminary analysis that evaluated various configurations, with the best performing one, that was ultimately utilized, detailed in Table I. As previously discussed, no crossover was used in our experiments [19]. As in [13, 14], no numeric constant was added to the terminal symbol set. In the experiments of Section IV-A, a phase in which the algorithm is executed with IGSM rate equal to 1 and DGSM rate equal to 0 is alternated with a phase in which the algorithm is executed with IGSM

TABLE I: Set of parameters used in this study.

Parameter	Value
Generations	1000
Population size	100
Elitism	True
Maximum initial depth	6
Function set	$\{+, -, \times, \div\}$
Selection algorithm	Tournament
Tournament size	2
Initialization method	Ramped Half-and-Half

rate equal to 0 and DGSM rate equal to 1. This switch between the operators happens after N generations, and various values of N are tested for all the studied variants: $N = 100, 250, 500, 750, 900$. These switch points are arbitrarily chosen to show the impact of DGSM at different moments of the evolutionary process. In the experiments of Sections IV-B and IV-C, instead, a more conventional version of SLIM is used, where IGSM and DGSM are used together. In those experiments, as in [13, 14], the IGSM rate is 0.3 (except for ld50 where it is 0.1 and concrete strength, where it is 0.5) and the DGSM rate is always 1 minus the IGSM probability.

The Monte Carlo cross-validation [17, 29] was applied using 30 independent runs, each with a different train-test partition, randomly drawn with 70% of the observations forming the training set and the remaining 30% the test set. The results we report in the next section are all medians over these 30 runs. GSGP shares the same hyperparameters, except that the probability of using DGSM is always set to 0. Fitness is assessed using the Root Mean Square Error (RMSE) between predicted and expected outputs. The implementation used for these experiments is available in the public domain library that we have implemented, that can be found at the following link: [anonymized](#).

IV. EXPERIMENTAL RESULTS

A. Isolating the Effects of IGSM and DGSM

Figure 2 and Figure 3 report the evolution of the test RMSE and elite size (defined as the number of nodes in the genotype of the best individual in the population), respectively. Results for training RMSE are provided in the Supplementary Material. In these plots, SLIM+2SIG is represented by a blue curve, SLIM*1SIG by a red curve and SLIM*ABS by a green curve. The version that switches to DGSM at generation N is represented by a dashed line. The version that always uses IGSM is represented by a solid line. The different columns of plots report the results for the different values of generation N (the generation where IGSM is replaced by DGSM). Finally, Table II and Table III report the IR_f and IR_s , respectively. In these tables, bold values indicate statistically significant results based on the Mann-Whitney U test, with significance level $\alpha = 0.05$ (the p -values returned by this statistical test can be found in the Supplementary Material). Also, a green color indicates that the version that introduces DGSM performs better than the version that only uses IGSM, while an orange color indicates the opposite. The darker the color, the more marked the result (an improvement of $m\%$ is represented by a darker color than an improvement of $n\%$,

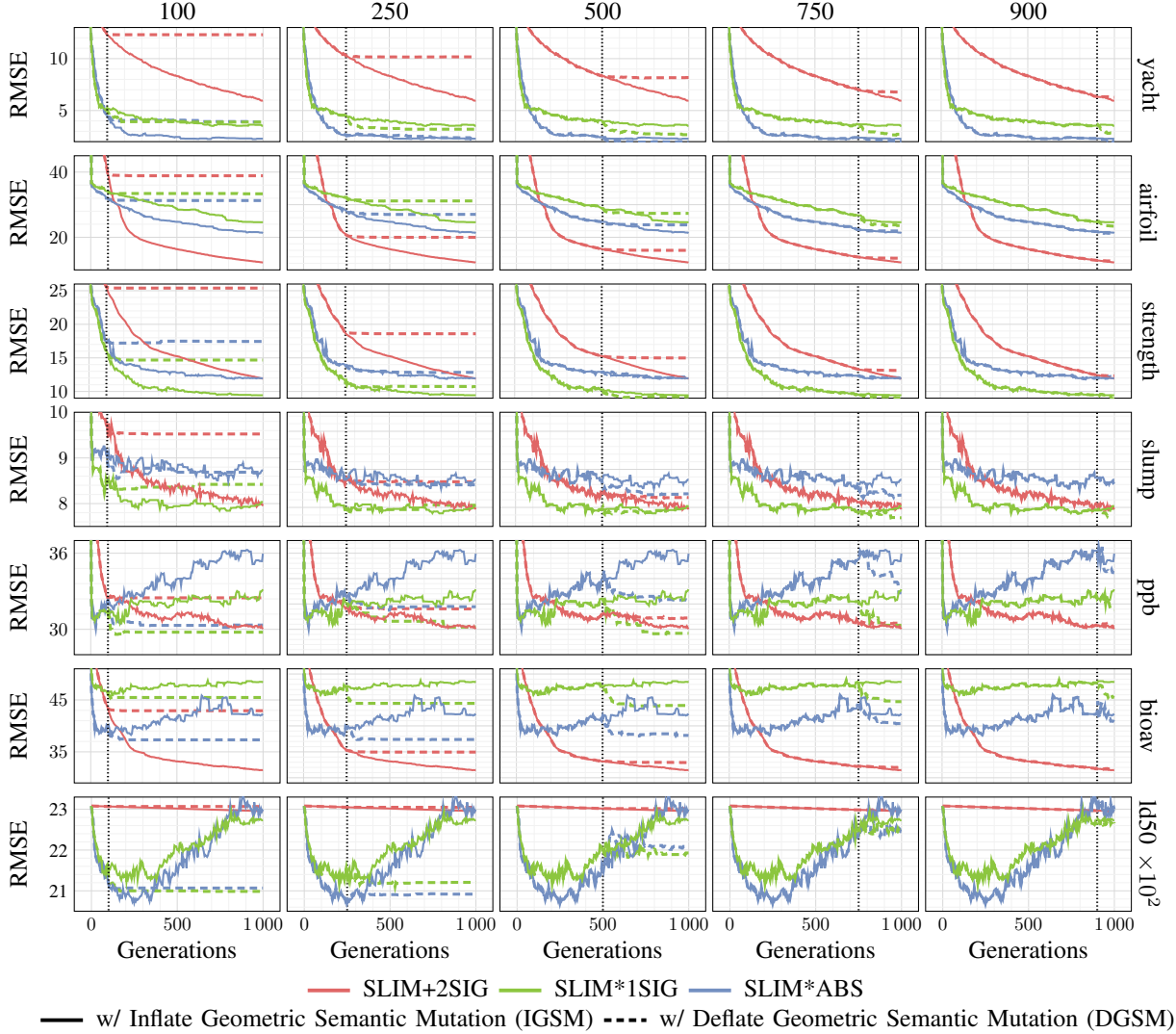


Fig. 2: Evolution of test fitness. Medians over 30 runs. Each column represents a different generation where IGSM is replaced by DGSM. That threshold generation is marked in the plots with a vertical dashed line.

if $m > n$).

Before delving into the detailed discussion of the results, it is immediately apparent from the tables that there is a dominant presence of green compared to other colors. This visual predominance clearly indicates the substantial effectiveness of the DGSM.

1) Influence of DGSM on SLIM+2SIG

For SLIM+2SIG, as expected, Figure 3 shows that applying DGSM, regardless of the generation N when it is introduced, significantly reduces individuals size. Interestingly, this reduction is achieved without compromising test fitness (Figure 2), which remains stable also after DGSM starts being applied. For $N = 900$, the decrease in the size of the individuals is even accompanied by an improvement in the test fitness. This suggests that DGSM either preserves the best individual, while removing unnecessary information, or finds an equally effective, but more compact, solution.

Our partial conclusions about the influence of DGSM on SLIM+2SIG are: (1) DGSM consistently achieves substantial reductions in model size, often without sacrificing accuracy; (2) switching to DGSM later in the run not only results

in smaller models but can also enhance their accuracy; (3) transitioning to DGSM does not result in overfitting across any of the test scenarios analyzed.

2) Influence of DGSM on SLIM*1SIG and SLIM*ABS

For SLIM*1SIG and SLIM*ABS, the introduction of DGSM not only produces smaller individuals compared to versions that use only IGSM, but also this size reduction often correlates with enhanced test fitness. Remarkably, across almost all datasets, there is a specific point where integrating DGSM not only improves test fitness, but also substantially exceeds the performance of the versions using only IGSM. An example is observed in the yacht dataset, where applying DGSM after 500 generations leads to significant performance gains. The optimal timing for introducing DGSM varies depending on the specific problem, particularly in relation to how the algorithms tend to overfit on those problems. For datasets where overfitting is less of a concern, implementing DGSM too early tends to be less beneficial; it reduces the size of individuals, but without enhancing performance. Our intuition is that, at earlier stages, the individuals are still relatively small, and only very few parts of the genotype can be removed

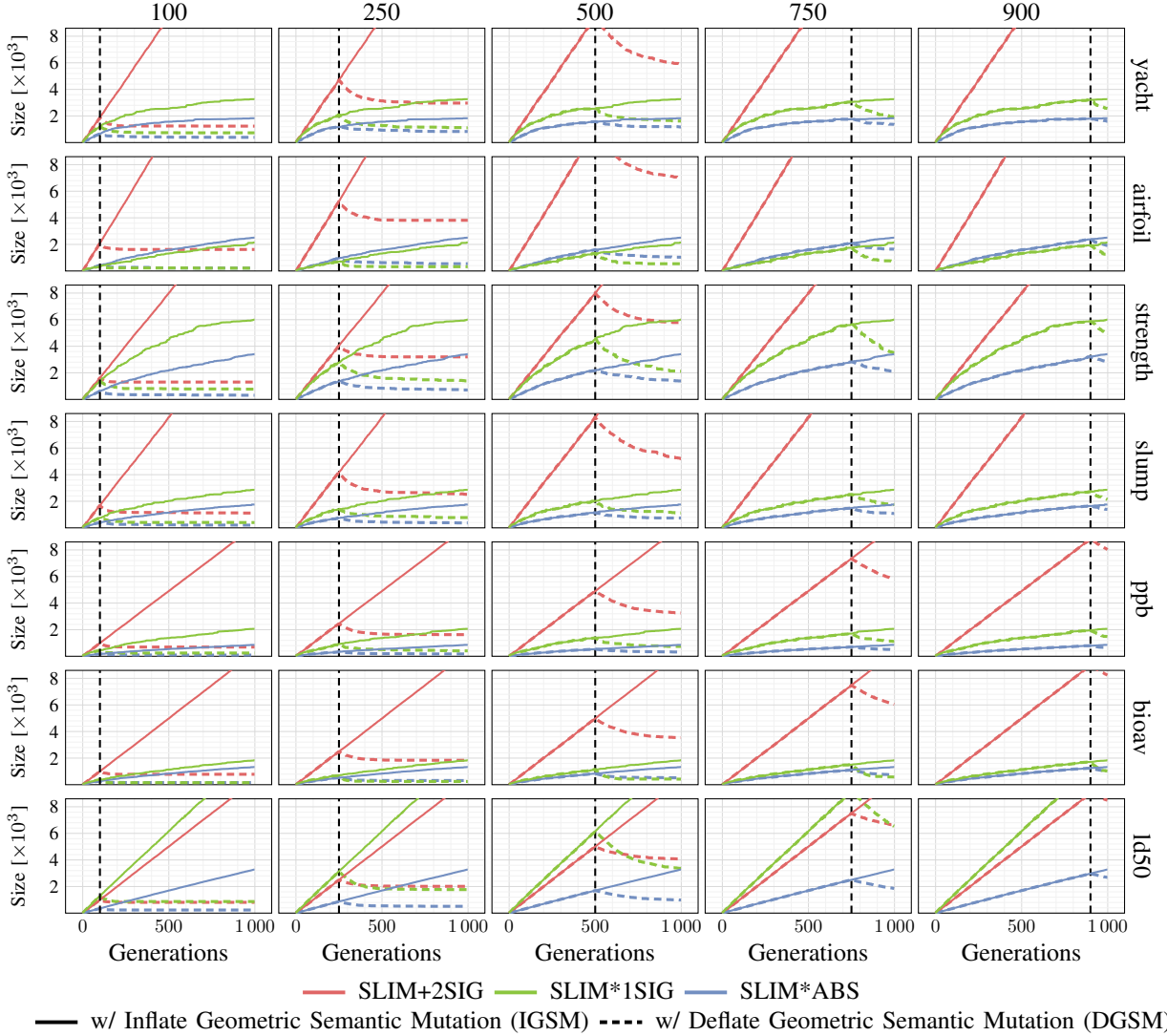


Fig. 3: Evolution of the best individual’s size (total tree nodes). Medians over 30 runs. Each column represents a different generation where IGSM is replaced by DGSM. That threshold generation is marked in the plots with a vertical dashed line.

without compromising the quality. Conversely, introducing DGSM later, specifically between generation 500 and 750, as consistently observed in our experiments, results in both a reduction in size and an improvement in test fitness. Also, if we introduce DGSM even later, only larger individuals are deflated. So, more iterations are required to achieve significant size reductions. However, the continued decrease in size, as shown in Figure 3, indicates that if we allowed the algorithms to run for more generations, we could obtain individuals that are as small as for other variants, but with superior performance compared to the versions that use only IGSM.

On the other hand, for problems where the versions that use only IGSM are affected by overfitting, like ppb, bioavailability and ld50, the introduction of DGSM is consistently effective in mitigating overfitting. For those problems, introducing DGSM early in the evolution is convenient since it not only preserves, but frequently enhances test fitness, while simultaneously reducing the size of the individuals (Table II).

Our findings can be better understood through key principles of GP. One such principle is the exploration-exploitation trade-off: early in evolution, broad exploration is crucial,

while later stages benefit from focused refinement. The deflate mutation mechanism in SLIM, which reduces the magnitude of changes, can hinder exploration if applied too early, limiting the algorithm’s ability to sample the search space effectively. On the other hand, applying it later supports adaptive control of variation by fine-tuning solutions in promising regions and reducing overfitting through parsimony pressure and bloat control. This approach also aligns with the building block hypothesis, as delayed deflation helps preserve and refine useful substructures rather than disrupting them prematurely. Overall, our results support the use of a time-adaptive deflate strategy to improve generalization while remaining consistent with core GP dynamics.

Our conclusions are that SLIM*1SIG and SLIM*ABS are the versions of SLIM that yielded the most favorable outcomes. For both these methods, introducing DGSM can be beneficial, if it is done at the right time. For problems susceptible to overfitting, introducing DGSM early in the evolutionary process proves advantageous. Conversely, in scenarios where overfitting is not an issue, it is beneficial to allow the algorithm to operate with IGSM for a more extended period before

TABLE II: Improvement rates in test fitness w.r.t the corresponding SLIM version that only uses IGSM. Green shades indicate improvement, while orange shades represent worsening. Each column represent a different generation after which only DGSM is applied. Bold values indicate statistical significance (Mann-Whitney U test, $\alpha = 0.05$).

Data	Algorithm	100	250	500	750	900
yacht	*1SIG	-0.71	0.02	0.13	0.1	0.1
	*ABS	-0.11	0.09	0.24	0.23	0.19
	+2SIG	-1.06	-0.7	-0.37	-0.13	0.05
airfoil	*1SIG	-0.47	-0.27	-0.12	0.03	0.0
	*ABS	-0.35	-0.27	-0.11	0.04	0.05
	+2SIG	-2.17	-0.63	-0.3	-0.11	0.04
slump	*1SIG	0.0	0.01	0.04	0.05	0.01
	*ABS	0.04	0.0	0.01	0.04	0.01
	+2SIG	-0.2	0.09	0.04	0.02	0.01
strength	*1SIG	-0.46	0.07	0.01	0.0	0.01
	*ABS	-0.54	-0.13	0.03	0.05	0.07
	+2SIG	-1.13	-0.56	-0.26	-0.1	0.03
ppb	*1SIG	0.16	0.12	0.1	0.09	0.04
	*ABS	0.1	0.09	0.1	0.09	0.05
	+2SIG	0.08	0.05	0.02	0.01	0.0
bioav	*1SIG	0.11	0.11	0.09	0.03	0.03
	*ABS	0.05	0.07	0.08	0.06	0.05
	+2SIG	-0.36	-0.11	0.05	0.01	0.01
ld50	*1SIG	0.08	0.09	0.04	0.02	0.0
	*ABS	0.08	0.07	0.04	0.01	0.0
	+2SIG	0.0	0.0	0.0	0.0	0.0

TABLE III: Improvement ratios in size, following the codification detailed in Table II.

Data	Algorithm	100	250	500	750	900
yacht	*1SIG	0.79	0.54	0.36	0.27	0.12
	*ABS	0.78	0.66	0.50	0.40	0.21
	+2SIG	0.93	0.84	0.68	0.42	0.18
airfoil	*1SIG	0.91	0.78	0.59	0.35	0.24
	*ABS	0.90	0.85	0.75	0.65	0.42
	+2SIG	0.92	0.82	0.67	0.43	0.18
slump	*1SIG	0.88	0.79	0.58	0.39	0.21
	*ABS	0.86	0.73	0.61	0.39	0.26
	+2SIG	0.93	0.85	0.68	0.41	0.19
strength	*1SIG	0.91	0.79	0.59	0.38	0.17
	*ABS	0.87	0.77	0.65	0.42	0.20
	+2SIG	0.92	0.80	0.64	0.41	0.18
ppb	*1SIG	0.89	0.80	0.62	0.42	0.24
	*ABS	0.88	0.80	0.64	0.46	0.28
	+2SIG	0.93	0.83	0.67	0.41	0.18
bioav	*1SIG	0.88	0.77	0.61	0.44	0.26
	*ABS	0.92	0.87	0.77	0.68	0.44
	+2SIG	0.92	0.82	0.64	0.39	0.18
ld50	*1SIG	0.93	0.84	0.70	0.44	0.18
	*ABS	0.93	0.85	0.72	0.46	0.20
	+2SIG	0.92	0.80	0.59	0.34	0.15

switching to DGSM.

B. Analysis of the Evolution of the TIE

The experiments using the TIE corroborate the insights discussed previously. Figure 4 shows the evolution of the TIE of both IGSM and DGSM, reporting median values across 30 runs. The TIE scores of IGSM show a consistent decline across generations, a finding that aligns with previous studies [15]. This trend underscores that IGSM is highly effective during the initial stages of the evolution, yet its impact tends to diminish in later generations. Conversely, the TIE of DGSM demonstrates an upward trend, indicating enhanced effectiveness as the evolutionary process progresses. In the later stages of the evolution, when individuals are larger, DGSM can be effectively applied. These larger individuals, with their extensive networks of neighbors, are likely to harbor

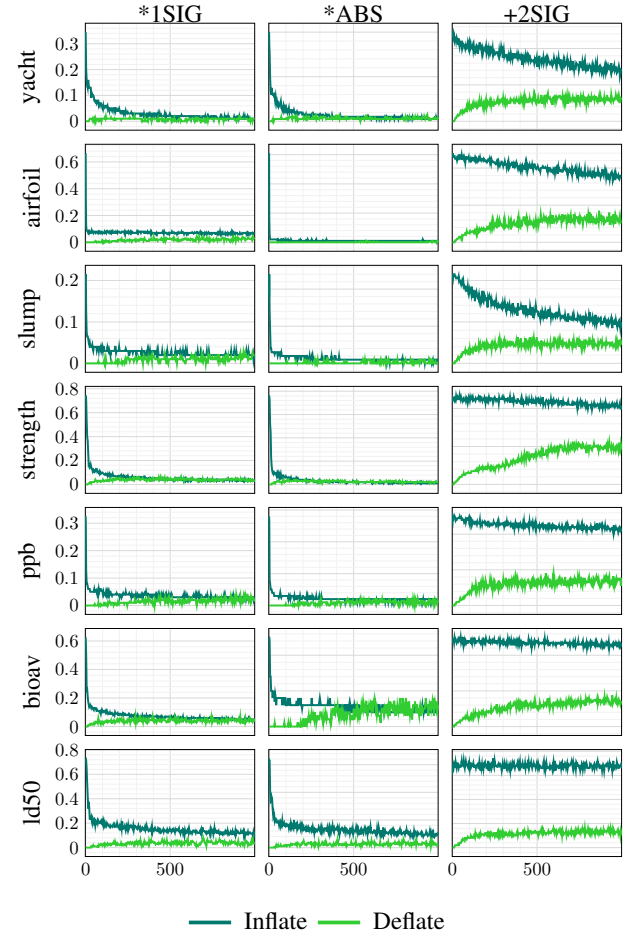


Fig. 4: Evolution of TIE, computed as in Section II-E. Medians over 30 runs for IGSM and DGSM operators are shown.

redundant or detrimental information. DGSM thus has the opportunity to remove such information, enhancing the model's generalization capability. This supports the notion that DGSM serves as an effective regularization technique within SLIM. Also, the fact that the results of the TIE corroborate those derived from the observation of the test RMSE plots holds significant importance. In fact, the TIE, unlike the emergence of overfitting, can be calculated during the evolution. So, the TIE can provide a reliable indicator for determining the optimal timing to switch between IGSM and DGSM.

C. Comparing SLIM with Other ML Techniques

In the literature, we can find several experimental comparisons between the traditional version of GSGP and some state-of-the-art Machine Learning (ML) algorithms (see for instance [30]). However, to the best of our knowledge, this type of comparison has never been extended to SLIM before. To fill this gap, and to assess the effectiveness of SLIM within the ML landscape, for each of the studied test problems we considered the top-performing SLIM variant that uses both IGSM and DGSM [13, 14] and we compared it against traditional ML methods. The SLIM+2SIG variant was chosen for the airfoil, bioavailability and ppb datasets, while SLIM*ABS performed best for slump and concrete strength and SLIM*1SIG yielded the best results for ld50. The parameters used are the same as in Table I. To ensure a fair

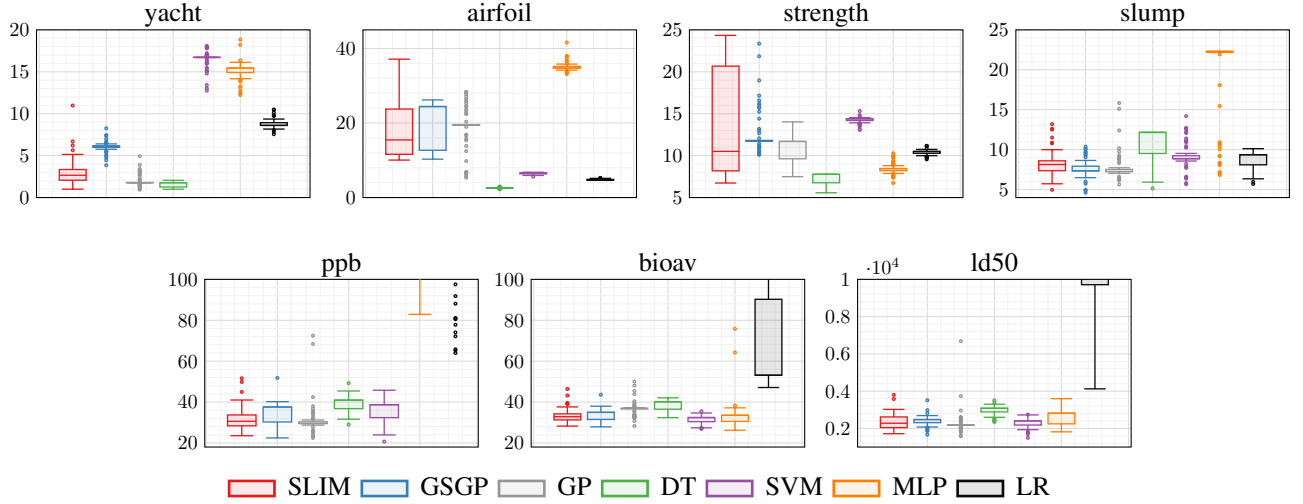


Fig. 5: Test fitness distribution of the best solutions across 30 runs.

comparison, we specifically excluded ensemble techniques, since SLIM does not fall into this category. Therefore, the algorithms selected for this comparison include Decision Trees (DTs) [31], Support Vector Machines (SVMs) [32], Multi-layer Perceptron (MLP) [33], and Linear Regression (LR) [34]. For all these algorithms, we have employed unmodified standard implementations from scikit-learn [35], guaranteeing replicability of the results. Furthermore, we have also added in the comparison traditional GSGP and a standard Koza-like version of GP (standard GP from now on). Following previous work [13, 14], besides using the parameters reported in Table I, for standard GP we have used standard subtree swapping crossover [36] and standard subtree mutation [36], with rates equal to 0.8 and 0.2 respectively, while limiting the maximum tree depth during the evolution to 17.

Figure 5 reports the box plots of the test fitness at termination returned by all these methods, based on results from 30 independent runs. Table IV presents a comparison of the sizes of the models generated by SLIM versus those produced by standard GP and GSGP. For all these results, the p -values derived from the Mann-Whitney U test are provided in the Supplementary Material. Examining Figure 5, we observe that DT outperforms the other algorithms on three out of seven datasets: yacht, airfoil, and concrete strength. In contrast, SLIM, GSGP, and standard GP each demonstrate superior performance on three different datasets. Specifically, they outperform the other studied algorithms in the slump, ppb (alongside SVM), and ld50 (together with SVM and MLP) datasets. Notably, SLIM significantly outperforms all competitors on the bioavailability dataset. Finally, we observe that SLIM consistently equals or exceeds GSGP in all datasets and outperforms standard GP across all but the yacht dataset, where standard GP returns the best results.

In the size comparison detailed in Table IV, we observe that SLIM consistently produces individuals that are smaller than those generated by GSGP. Additionally, SLIM creates individuals that are smaller or of comparable size to those produced by standard GP in four of the seven datasets: yacht, airfoil, slump, and concrete strength. However, in the remain-

TABLE IV: Median size at the last generation of the SLIM, GP and GSGP best individuals. The values in the GP and GSGP columns are bold when the difference with SLIM is statistically significant (Mann Whitney U test with $\alpha = 0.05$).

Dataset	SLIM	GP	GSGP
yacht	914	685	18592
airfoil	634.5	1791	21100
slump	728	990	16330
strength	1718	1004	15979.5
ppb	387	63	9770
bioav	692	261	9966
ld50	652	113	9999.5

ing datasets, i.e., ppb, bioavailability and ld50, standard GP generates individuals that are smaller than those produced by SLIM.

Overall, our findings suggest that SLIM represents a highly promising new direction in genetic programming. It not only achieves excellent accuracy, competitive with those of established machine learning algorithms, but also frequently produces models that are compact.

V. CONCLUSIONS AND FUTURE WORK

With the introduction of the novel Deflate Geometric Semantic Mutation (DGSM), the Semantic Learning algorithm based on Inflate and deflate Mutation (SLIM) enriches Geometric Semantic Genetic Programming (GSGP) with the capability to produce compact models, thus advancing our ambitions towards evolutionary interpretable AI. This paper presents the first thorough investigation into the functionality and tangible benefits of DGSM. In the initial phase of our experimental study, we alternated between phases where only the traditional Inflate Geometric Semantic Mutation (IGSM) was used and phases where IGSM was substituted with DGSM alone. This approach helped us effectively isolate and better understand the distinct impacts of each mutation on the algorithm. Our findings provide clear insights, demonstrating that DGSM, when introduced at the appropriate stage of the evolution, can contribute to the creation of models that are not only as accurate or better than those evolved through traditional GSGP, but also as compact or smaller than the models evolved by standard Genetic Programming (GP). The optimal timing for introducing DGSM varies depending on

the specific problem; early introduction is advantageous for datasets susceptible to overfitting, whereas in scenarios where overfitting is less of a concern, it is beneficial to employ IGSM for a longer period before switching to DGSM. These results underscore the critical role of DGSM as a regularization factor within the algorithm's dynamics, a role that is crucial for the future development of models that are general, robust, and interpretable. The validity of these outcomes is strengthened by different evidence from various experimental approaches, including (1) a comparison of solely IGSM-based versions with those introducing DGSM at different stages, (2) an analysis of the improvement rate, which quantifies the benefits of incorporating DGSM, and (3) the Training Improvement Effectiveness (TIE). The consistency of findings across these diverse methodologies lends robustness to our conclusions. Additionally, our comparative analysis of SLIM against well-established machine learning algorithms confirms that SLIM delivers competitive results, reinforcing its status as a promising approach. This encourages further exploration and development of SLIM.

Building on the insights garnered from our study, there are several promising avenues for future research that warrant exploration. A key area involves developing an adaptive strategy that dynamically determines the optimal timing for applying inflate or deflate mutations [37, 38]. The results we have presented demonstrate that the TIE can be an effective indicator for guiding the algorithm in managing the probabilities of applying IGSM and DGSM. Currently, the blocks removed using DGSM are selected at random. Future work should aim to develop a more sophisticated deflation criterion, specifically one that determines which elements to remove from the linked list structure. A recent study introduced a new geometric semantic crossover operator that takes advantage of the linked list structure [39]. Future research should extend the analysis presented in this work to include recombination operators.

Acknowledgments. This work was supported by national funds through FCT (Fundação para a Ciência e a Tecnologia), under the project - UIDB/04152 - Centro de Investigação em Gestão de Informação (MagIC)/NOVA-IMS. The paper is based upon work from a scholarship supported by SPECIES (<http://species-society.org>), the Society for the Promotion of Evolutionary Computation in Europe and its Surroundings.

REFERENCES

- [1] Alberto Moraglio, Krzysztof Krawiec, and Colin G. Johnson. Geometric Semantic Genetic Programming. In Carlos A. Coello Coello et al., editor, *Parallel Problem Solving from Nature*, volume 7491 of *LNCS*, pages 21–31. Springer, 2012.
- [2] Riccardo Poli, William B. Langdon, and Nicholas F. Mcphee. *A field guide to genetic programming*. 2008.
- [3] Pedro G Espejo, Sebastián Ventura, and Francisco Herrera. A survey on the application of genetic programming to classification. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 40(2):121–144, 2009.
- [4] Wolfgang Banzhaf, JR Koza, C Ryan, L Spector, and C Jacob. Genetic programming. *IEEE Intelligent Systems and their Applications*, 15(3):74–84, 2000.
- [5] Leonardo Vanneschi, Mauro Castelli, Luca Manzoni, and Sara Silva. A new implementation of geometric semantic GP and its application to problems in pharmacokinetics. In *Genetic Programming. EuroGP 2013*, pages 205–216. Springer, 2013.
- [6] Xianshun Chen, Yew-Soon Ong, Meng-Hiot Lim, and Kay Chen Tan. A multi-facet survey on memetic computation. *IEEE Transactions on evolutionary computation*, 15(5):591–607, 2011.
- [7] Leonardo Trujillo, Emigdio Z-Flores, Perla S Juárez-Smith, Pierrick Legrand, Sara Silva, Mauro Castelli, Leonardo Vanneschi, Oliver Schütze, and Luis Muñoz. Local search is underused in genetic programming. *Genetic Programming Theory and Practice XIV*, pages 119–137, 2018.
- [8] Gloria Pietropolli, Luca Manzoni, Alessia Paoletti, and Mauro Castelli. Combining Geometric Semantic GP with gradient-descent optimization. In *Genetic Programming. EuroGP 2022*, pages 19–33. Springer, 2022.
- [9] Lorenzo Bonin, Luigi Rovito, Andrea De Lorenzo, and Luca Manzoni. Cellular geometric semantic genetic programming. *Genetic Programming and Evolvable Machines*, 25(1):8, 2024.
- [10] Giorgia Nadizar, Berfin Sakallioğlu, Fraser Garrow, Sara Silva, and Leonardo Vanneschi. Geometric semantic GP with linear scaling: Darwinian versus Lamarckian evolution. *Genetic Programming and Evolvable Machines*, 25(2):1–24, 2024.
- [11] Mauro Castelli, Leonardo Trujillo, and Leonardo Vanneschi. Energy Consumption Forecasting Using Semantic-Based Genetic Programming with Local Search Optimizer. *Computational intelligence and neuroscience*, 2015(1):971908, 2015.
- [12] Francesco Marchetti, Gloria Pietropolli, Federico Julian Camerota Verdù, Mauro Castelli, and Edmondo Minisci. Automatic design of interpretable control laws through parametrized Genetic Programming with adjoint state method gradient evaluation. *Applied Soft Computing*, 159:111654, 2024.
- [13] Leonardo Vanneschi. SLIM_GSGP: The Non-bloating Geometric Semantic Genetic Programming. In Mario Giacobini, Bing Xue, and Luca Manzoni, editors, *Genetic Programming. EuroGP 2024*, pages 125–141. Springer Nature, 2024.
- [14] Leonardo Vanneschi, Davide Farinati, Diogo Rasteiro, Liah Rosenfeld, Gloria Pietropolli, and Sara Silva. Exploring Non-bloating Geometric Semantic Genetic Programming. *Genetic Programming Theory and Practice XXI*, pages 237–258, 2025.
- [15] Ivo Gonçalves, Sara Silva, Carlos M. Fonseca, and Mauro Castelli. Unsure when to stop? In *Proceedings of the Genetic and Evolutionary Computation Conference*. ACM, jul 2017. doi: 10.1145/3071178.3071328.
- [16] Davide Farinati and Leonardo Vanneschi. A study

- of dynamic populations in geometric semantic genetic programming. *Information Sciences*, 648:119513, 2023.
- [17] Leonardo Vanneschi and Sara Silva. *Lectures on Intelligent Systems*. Natural Computing Series. Springer, 2023. doi: 10.1007/978-3-031-17922-8.
- [18] Leonardo Vanneschi. *An Introduction to Geometric Semantic Genetic Programming*, pages 3–42. Springer International Publishing, Cham, 2017.
- [19] Mauro Castelli, Luca Manzoni, Ivo Gonçalves, Leonardo Vanneschi, Leonardo Trujillo, and Sara Silva. An Analysis of Geometric Semantic Crossover: A Computational Geometry Approach. In *8th Int. Conf. on Evol. Computation Theory and Applications*, pages 201–208, 01 2016.
- [20] Leonardo Vanneschi, Sara Silva, Mauro Castelli, and Luca Manzoni. Geometric Semantic Genetic Programming for Real Life Applications. In Rick Riolo et al., editor, *Genetic Programming Theory and Practice XI*, Genetic and Evolutionary Computation. Springer, 2013.
- [21] Ivo Gonçalves, Sara Silva, and Carlos M. Fonseca. On the Generalization Ability of Geometric Semantic Genetic Programming. In Penousal et al. Machado, editor, *Genetic Programming*, pages 41–52. Springer, 2015.
- [22] Illya Bakurov, José Manuel Muñoz Contreras, Mauro Castelli, Nuno Rodrigues, Sara Silva, Leonardo Trujillo, and Leonardo Vanneschi. Geometric semantic genetic programming with normalized and standardized random programs. *Genetic Programming and Evolvable Machines*, 25(1), 2024.
- [23] Francesco Archetti, Stefano Lanzeni, Enza Messina, and Leonardo Vanneschi. Genetic programming for computational pharmacokinetics in drug discovery and development. *Genetic Programming and Evolvable Machines*, 8(4):413–432, 2007.
- [24] Sara Silva and Leonardo Vanneschi. Operator equalisation, bloat and overfitting: a study on human oral bioavailability prediction. In Franz Rothlauf, editor, *Genetic and Evolutionary Computation Conference, GECCO 2009*, pages 1115–1122. ACM, 2009.
- [25] Mauro Castelli, Leonardo Vanneschi, and Sara Silva. Prediction of high performance concrete strength using Genetic Programming with geometric semantic genetic operators. *Expert Systems with Applications*, 40(17):6856 – 6862, 2013.
- [26] James McDermott, David R. White, Sean Luke, Luca Manzoni, Mauro Castelli, Leonardo Vanneschi, Wojciech Jaśkowski, Krzysztof Krawiec, Robin Harper, Kenneth De Jong, and Una-May O’Reilly. Genetic Programming Needs Better Benchmarks. In *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation, GECCO ’12*, pages 791–798, New York, NY, USA, 2012. ACM. doi: 10.1145/2330163.2330273.
- [27] Gloria Pietropolli, Luca Manzoni, Alessia Paoletti, and Mauro Castelli. On the hybridization of geometric semantic GP with gradient-based optimizers. *Genetic Programming and Evolvable Machines*, 24(2):16, 2023.
- [28] Gloria Pietropolli, Stefano Nichele, and Eric Medvet. The role of the substrate in ca-based evolutionary algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 768–777, 2024.
- [29] Werner Dubitzky, Martin Granzow, and Daniel P. Berrar. *Fundamentals of Data Mining in Genomics and Proteomics*. Springer, 2006. ISBN 0387475087.
- [30] Jose Manuel Muñoz Contreras, Leonardo Trujillo, Daniel E. Hernandez, and Luis A. Cardenas Florido. Benchmarking GSGP: Still competitive 10 years later? *Genet. Program. Evolvable Mach.*, 26(1):6–18, 2024.
- [31] Lior Rokach and Oded Maimon. Decision trees. *Data mining and knowledge discovery handbook*, pages 165–192, 2005.
- [32] Shan Suthaharan and Shan Suthaharan. Support vector machine. *Machine learning models and algorithms for big data classification: thinking with examples for effective learning*, pages 207–235, 2016.
- [33] Allan Pinkus. Approximation theory of the MLP model in neural networks. *Acta numerica*, 8:143–195, 1999.
- [34] Douglas C Montgomery, Elizabeth A Peck, and G Geoffrey Vining. *Introduction to linear regression analysis*. John Wiley & Sons, 2021.
- [35] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *the Journal of machine Learning research*, 12:2825–2830, 2011.
- [36] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, USA, 1992.
- [37] José Ferreira, Mauro Castelli, Luca Manzoni, and Gloria Pietropolli. A Self-Adaptive Approach to Exploit Topological Properties of Different GAs’ Crossover Operators. In *European Conference on Genetic Programming (Part of EvoStar)*, pages 3–18. Springer, 2023.
- [38] Andres Felipe Cruz-Salinas and Jonatan Gomez Perdomo. Self-adaptation of genetic operators through genetic programming techniques. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 913–920, 2017.
- [39] Gloria Pietropolli, Davide Farinati, Luca Manzoni, Mauro Castelli, Sara Silva, and Leonardo Vanneschi. Introducing Crossover in SLIM-GSGP. In *European Conference on Genetic Programming (Part of EvoStar)*, pages 103–119. Springer, 2025.